

Workshop-Notizen

Datum: 14.07.2026 | Dauer: 3 Stunden | Ort: Büro München, Schulungsraum 2 | Trainer: Markus Feldmann (extern)

Heute fand der interne Workshop „Clean Code in der Praxis“ statt. Ziel war es, dem Entwicklungsteam praxisnahe Prinzipien für lesbaren, wartbaren Code zu vermitteln und anhand von Beispielen aus unserer eigenen Codebasis zu diskutieren, wo wir aktuell von Clean-Code-Prinzipien abweichen.

Teilnehmer

Julia Berger (Backend), Tom Kessler (Backend), Nina Voss (Frontend), Ali Yildiz (Frontend), Sven Lorenz (QA), Markus Feldmann (Trainer, extern)

Agenda

- Einführung: Warum Clean Code? Kosten von technischer Schuld
- Benennung von Variablen, Funktionen und Klassen
- Funktionsgröße und Single Responsibility Principle
- Kommentare: wann sinnvoll, wann Code-Smell
- Fehlerbehandlung und Exceptions
- Live-Refactoring einer realen Klasse aus dem Billing-Service
- Diskussion: eigene Coding-Guidelines anpassen?

Wichtige Inhalte

Markus stellte anhand von Kent Becks vier Regeln für einfachen Code (Tests bestehen, Absicht klar ausdrücken, keine Duplikation, minimale Elemente) den roten Faden des Workshops vor. Besonders diskutiert wurde die Namensgebung: Funktionsnamen sollten ein Verb enthalten und eine einzige, klar erkennbare Aufgabe beschreiben (z. B. `calculateInvoiceTotal` statt `process`).

Beim Thema Funktionsgröße wurde die Faustregel „eine Funktion sollte auf einen Bildschirm passen“ vorgestellt. Am Beispiel der Klasse `InvoiceService` aus unserem Billing-Service (aktuell 340 Zeilen, 1 Klasse) wurde gemeinsam ein Refactoring-Vorschlag erarbeitet: Aufteilung in `InvoiceCalculator`, `InvoiceValidator` und `InvoicePersistence`.

Zur Fehlerbehandlung riet Markus, generische `catch(Exception e)`-Blöcke zu vermeiden und stattdessen spezifische Exception-Typen zu verwenden sowie Fehler so früh wie möglich (fail fast) zu erkennen.

Ergebnisse & Learnings

- Das Team einigt sich darauf, künftig Code-Reviews stärker auf Funktionsgröße und Namensgebung zu prüfen.
- Die Klasse `InvoiceService` wird als erstes konkretes Refactoring-Beispiel im nächsten Sprint angegangen.
- Generische Exception-Blöcke sollen schrittweise durch spezifische Exceptions ersetzt werden.

Offene Fragen

- Sollen wir ein gemeinsames Linter-Regelwerk (z. B. Checkstyle/ESLint) teamübergreifend verbindlich machen?
- Wie gehen wir mit Altcode um, der aus Zeitgründen nicht refactored werden kann?

Action Items

- Tom Kessler: Refactoring-Vorschlag für InvoiceService bis 21.07.2026 als Pull Request einreichen.
- Nina Voss: ESLint-Regelwerk für das Frontend-Team bis Ende des Sprints vorschlagen.
- Sven Lorenz: Checkliste für Code-Reviews um Clean-Code-Kriterien ergänzen.